

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS



COMPILADORES Y TEORÍA DE LENGUAJE

“Compilador - Intérprete para un Mini Lenguaje de programación en Español enfocado en Matemática”

DOCENTE: MANUEL YURI APAZA VALENCIA

INTEGRANTES:

Serrano Yanapa, Pedro Czar Paolo
ORCID: [0009-0007-8326-8959](https://orcid.org/0009-0007-8326-8959)

2024 - 119023

Pacci Perez, Giorgini Disalvatore
ORCID: [0009-0000-7031-7140](https://orcid.org/0009-0000-7031-7140)

2024 - 119035

SEMESTRE ACADÉMICO: 5TO SEMESTRE

17 / 07 / 26

TACNA - PERÚ

A través del presente informe, presentamos el desarrollo de nuestro proyecto final. En primer lugar, ofrecemos nuestras sinceras disculpas por cualquier inconveniente o demora ocasionada en la entrega del mismo. Asimismo, deseamos expresar nuestro profundo agradecimiento al estimado Ing. Manuel Apaza Valencia. Gracias a su metodología didáctica, su paciencia y su nivel de exigencia, hemos logrado culminar con éxito el desarrollo de este interesante Compilador-Intérprete.

ÍNDICE

1. Introducción.....	4
1. Características Sintácticas.....	6
2. Restricciones del Lenguaje.....	6
3. Especificación Léxica.....	7
Palabras reservadas.....	7
Clases de Tokens.....	7
• Constantes e Identificadores.....	8
• Operadores Aritméticos.....	8
• Operadores Relacionales y Asignación.....	9
• Delimitadores.....	9
4. Expresiones Regulares.....	10
5. Diagrama del Autómata Finito.....	11
6. Analizador Léxico.....	11
1. Configuración y Expresiones Regulares Base.....	12
2. Reglas de Producción Léxica.....	13
3. Tabla Completa de Tokens.....	13
4. Detección y Manejo de Errores Léxicos.....	14
7. Gramática del Lenguaje.....	15
8. Eliminación de Ambigüedad.....	16
A. Factorización por la Izquierda.....	16
B. Eliminación de Recursividad por la Izquierda.....	17
C. Conversión LL(1) (Gramática Final Optimizada).....	17
9. Árbol Sintáctico.....	19
10. Analizador Sintáctico Implementado.....	20
1. Implementación con Bison: parser.y.....	20
2. Funcionamiento Interno (Shift/Reduce).....	20
3. Precedencia y Tipos.....	21
4. Gramática y Construcción del AST.....	21
5. Ejercicios implementados obteniendo el AST en el programa.....	22
11. Tabla de Símbolos.....	24
12. Analizador Semántico.....	24
1. Validaciones Implementadas.....	24
2. Ejemplo de Validación Semántica.....	25
13. Código Intermedio.....	26
14. Optimización de Código.....	27
15. Generación de Código Final.....	27
16. Implementación.....	28
Lenguaje Utilizado.....	28
Herramientas Utilizadas.....	29

17. Arquitectura del Sistema.....	29
Descripción de las Fases.....	30
18. Interfaz del Programa.....	31
19. Resultados.....	32
20. Conclusiones.....	32
21. Recomendaciones.....	33
22. Bibliografía.....	34
23. Anexo.....	34

1. Introducción

Un compilador es un sistema de software complejo encargado de procesar un programa escrito en un lenguaje fuente y transformarlo en una representación válida para su ejecución o traducción posterior. Este proceso no se limita al simple reconocimiento de palabras, sino que integra una arquitectura de múltiples fases interconectadas: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio, optimización y generación de código final.

Descripción del problema

El aprendizaje de la teoría de compiladores suele presentar una curva de dificultad pronunciada, agravada por el hecho de que la mayoría de los lenguajes de programación comerciales utilizan sintaxis en inglés y requieren librerías externas complejas para realizar cálculos matemáticos simbólicos. Existe la necesidad de una herramienta pedagógica que reduzca la carga cognitiva del idioma y que, al mismo tiempo, integre operaciones matemáticas avanzadas de forma nativa para facilitar la asimilación conjunta de la lógica de programación y el cálculo.

Importancia de los compiladores e intérpretes

Comprender el diseño y la construcción de compiladores es un pilar fundamental en la formación en Ingeniería en Informática y Sistemas. Estos programas son la columna vertebral que permite traducir la abstracción del pensamiento lógico humano a instrucciones que una máquina puede procesar. Desarrollar un compilador propio otorga una visión profunda sobre la gestión de memoria, la estructura de datos, la optimización de algoritmos y el funcionamiento interno de los lenguajes de programación.

OBJETIVOS DEL PROYECTO:

Objetivo General:

El objetivo principal de este proyecto es permitir a los estudiantes escribir programas estructurados utilizando una sintaxis familiar e intuitiva en idioma español, reduciendo la barrera del idioma extranjero al momento de interiorizar la teoría de lenguajes y autómatas.

Objetivos Específicos:

- **Implementar el análisis léxico y semántico:** Integrar el analizador léxico con Flex y el analizador sintáctico con Bison en base a nuestro compilador
- **Construir el análisis semántico:** Verificar reglas de la gramática de lenguaje del programa con uso de identificadores, tipos de caracteres y validez de delimitadores e instrucciones.
- **Generar código intermedio:** Transforma las instrucciones a código máquina para su posterior procedimiento.
- **Optimizar el código generado:** Aplicará reducción de sentencias, simplificación algebraica y eliminación de código muerto.
- **Validar el resultado final:** Ejecutar el programa con diferentes ejercicios que demostraran el correcto funcionamiento de cada una de las fases implementadas.

ALCANCES Y LIMITACIONES

Alcances del Proyecto

El alcance del compilador abarca desde la lectura del archivo fuente hasta la generación de un ejecutable final en Windows. El proyecto soporta tipos de datos básicos, estructuras de control de flujo estandarizadas y operaciones de entrada/salida. Como principal innovación, alcanza la resolución de cálculo simbólico básico (derivadas e integrales indefinidas de polinomios simples)

Limitaciones del Proyecto

La principal limitación del proyecto es que el lenguaje requiere de un compilador secundario subyacente (GNU GCC) para generar el archivo binario ejecutable (.exe), ya que no emite código máquina ni ensamblador de forma directa por el motivo de implementar adicionalmente un interprete y pueda ambos trabajar de forma similar.

Respecto al Lenguaje de programación Craft se tiene las siguientes limitaciones:

- **Paradigma de programación:** El compilador no realiza programación orientada a objetos (POO).
- **Datos de memoria y diseño:** La gramática no implementa punteros direccionales o incluso memoria de flujo estática.
- **Generación de código final:** En esta fase se realiza mas a una generación de codigo de nivel C++ o incluso a un compilador GNU C++ (g++).

1. Características Sintácticas

La sintaxis de craft sigue un estilo imperativo y estructurado similar a C++, pero con palabras reservadas íntegramente en español. Su principal diferenciador es la inclusión de construcciones nativas de primer nivel para el cálculo matemático simbólico. A continuación se presenta el resumen de sus construcciones:

Tabla 01. Características sintácticas de nuestro lenguaje de programación

Categoría	Construcción	Ejemplo en craft
Tipos de datos	Entero, Decimal, Texto, Función matemática	entero x = 5; funcion_matematica f;
Control de flujo	si/sino	si (x > 0) { ... } sino { ... }
Bucles	mientras, para	mientras (i < 10) { x = x + 1; }
Funciones Core (Matemáticas)	derivar, integrar_indefinida	d = derivar(f, x);
Entrada/Salida	imprimir, leer	imprimir("Resultado: ", d);
Operadores aritméticos	+, -, *, /, ^	y = a * x ^ 2 + b;
Operadores relacionales	==, !=, <, >, <=, >=	si (a != b) { ... }
Operadores lógicos	y, o, no	si (a > 0 y b < 5) { ... }

Fuente. Elaboración propia

2. Restricciones del Lenguaje

- Los identificadores deben comenzar obligatoriamente con una letra o dígito.
- El tipo de dato `funcion_matematica` es de uso exclusivo para el almacenamiento de expresiones algebraicas simbólicas; no opera como una variable numérica convencional que pueda evaluarse aritméticamente en tiempo de ejecución.

- Las operaciones nativas de cálculo (derivar e integrar_indefinida) son estrictas en su firma: requieren exactamente dos parámetros correspondientes a identificadores válidos (la función a evaluar y la variable respecto a la cual se aplicará la operación).
- La validación semántica aplica un tipado estricto. No se permiten asignaciones implícitas entre cadenas de texto (texto) y tipos numéricos (entero, decimal) sin una conversión previa.
- En esta versión base orientada al aprendizaje, no se soporta la declaración de arreglos (arrays) ni estructuras complejas de Programación Orientada a Objetos (POO), manteniendo un modelo de memoria estático y secuencial.

3. Especificación Léxica

El análisis léxico es la primera fase del compilador. En esta etapa, la secuencia de caracteres del código fuente se agrupa en unidades lógicas con significado, denominadas tokens. A continuación, se detallan los componentes léxicos identificados por el analizador construido con Flex.

Palabras reservadas

Las palabras reservadas de craft conforman el núcleo estructural e imperativo del lenguaje, así como sus capacidades nativas de cálculo simbólico. Estas palabras tienen un significado predefinido y no pueden ser utilizadas como identificadores de variables o funciones.

Tabla 02. Palabras reservadas de craft

entero	decimal	texto
si	sino	mientras
retornar	imprimir	leer
y	o	integrar_indefinida
derivar	para	funcion_matematica

Fuente. Elaboración propia

Clases de Tokens

El analizador léxico reconoce y clasifica los siguientes patrones a través de expresiones regulares. Se dividen en constantes, identificadores, operadores matemáticos/relacionales y delimitadores de bloque.

- **Constantes e Identificadores**

Las constantes son valores inmutables que se evalúan en tiempo de compilación y no pueden modificarse a lo largo de la ejecución del programa, mientras que los identificadores son los nombres definidos por el programador para representar variables, funciones, clases y etiquetas en un programa.

Tabla 03. Constantes e Identificadores usados para nuestro lenguaje

Clase	Expresión Regular	Token	Ejemplo en craft
Entero	[0-9]+	NUM_ENTERO	42
Decimal	[0-9]+\.[0-9]+	NUM_DECIMAL	3.1415
Cadena de texto	\("[^"]*\")	CADENA	"La derivada es: "
Identificador	[a-zA-Z_][a-zA-Z0-9_]*	ID	variable_x

Fuente. Elaboración propia

- **Operadores Aritméticos**

Son identificadores predeterminados que obtienen solo operadores del código fuente.

Tabla 04. Operadores Aritméticos usados para nuestro lenguaje

Clase	Expresión Regular	Token	Ejemplo en craft
Suma	\+	MAS	+
Resta	-	MENOS	-
Multiplicación	*	POR	*
División	/	DIV	/
Potencia	\^	POTENCIA	^

Fuente. Elaboración propia

- **Operadores Relacionales y Asignación**

Los operadores de asignación guardan valores en variables, mientras que los relacionales comparan dos datos, devolviendo un resultado lógico

Tabla 05. Operadores Relacionales y Asignación usados para nuestro lenguaje

Clase	Expresión Regular	Token	Ejemplo en craft
Asignación	=	ASIGNAR	=
Igualdad	==	EQUALS	==
Diferencia	!=	NEQUALS	!=
Mayor que	>	MAYOR	>
Menor que	<	MENOR	<

Fuente. Elaboración propia

- **Delimitadores**

Son caracteres o símbolos especiales que indican el inicio, fin o separación de elementos en el código fuente.

Tabla 06. Delimitadores usados para nuestro lenguaje

Clase	Expresión Regular	Token	Ejemplo en craft
Punto y coma	;	PUNTOCOMA	;
Coma	,	COMA	,
Paréntesis Izquierdo	\(	PARENIZQ	(
Paréntesis Derecho	\)	PARENDER	)
Llave Izquierda	\{	LLAVEIZQ	{
Llave Derecha	\}	LLAVEDER	}

Fuente. Elaboración propia

4. Expresiones Regulares

Las categorías léxicas del lenguaje craft se definen formalmente mediante Expresiones Regulares (ER). Durante el proceso de compilación, la herramienta Flex convierte internamente estas ER en un Autómata Finito No Determinista (NFA) utilizando el algoritmo de construcción de Thompson. Posteriormente, el sistema aplica la construcción de subconjuntos para transformar dicho modelo en un Autómata Finito Determinista (DFA) equivalente y optimizado.

El DFA generado por nuestro analizador léxico es capaz de procesar las secuencias de caracteres del archivo fuente en un tiempo $O(n)$, el cual es proporcional a la longitud de la cadena de entrada, operando de manera lineal y sin retroceder nunca durante la lectura.

1. Identificadores (ID)

Un identificador válido debe comenzar obligatoriamente con una letra o un guion bajo, seguido de la cerradura de Kleene (cero o más repeticiones) de letras, dígitos o guiones bajos.

$$\text{LETRA} \cdot (\text{LETRA} \mid \text{DIGITO} \mid \text{GUION})^* = [\text{a-z A-Z_}][\text{a-z A-Z 0-9_}]^*$$

2. Números Enteros (NUM_ENTERO)

Se define como una concatenación de al menos un dígito, seguido de cero o más dígitos adicionales.

$$\text{DIGITO}^+ = [0-9][0-9]^*$$

3. Números Decimales (NUM_DECIMAL)

Se compone de la cerradura positiva de dígitos, concatenada con el símbolo literal del punto decimal, y finaliza con otra cerradura positiva de dígitos.

$$\text{DIGITO}^+ \cdot '.' \cdot \text{DIGITO}^+ = [0-9]^+ \cdot '.' [0-9]^+$$

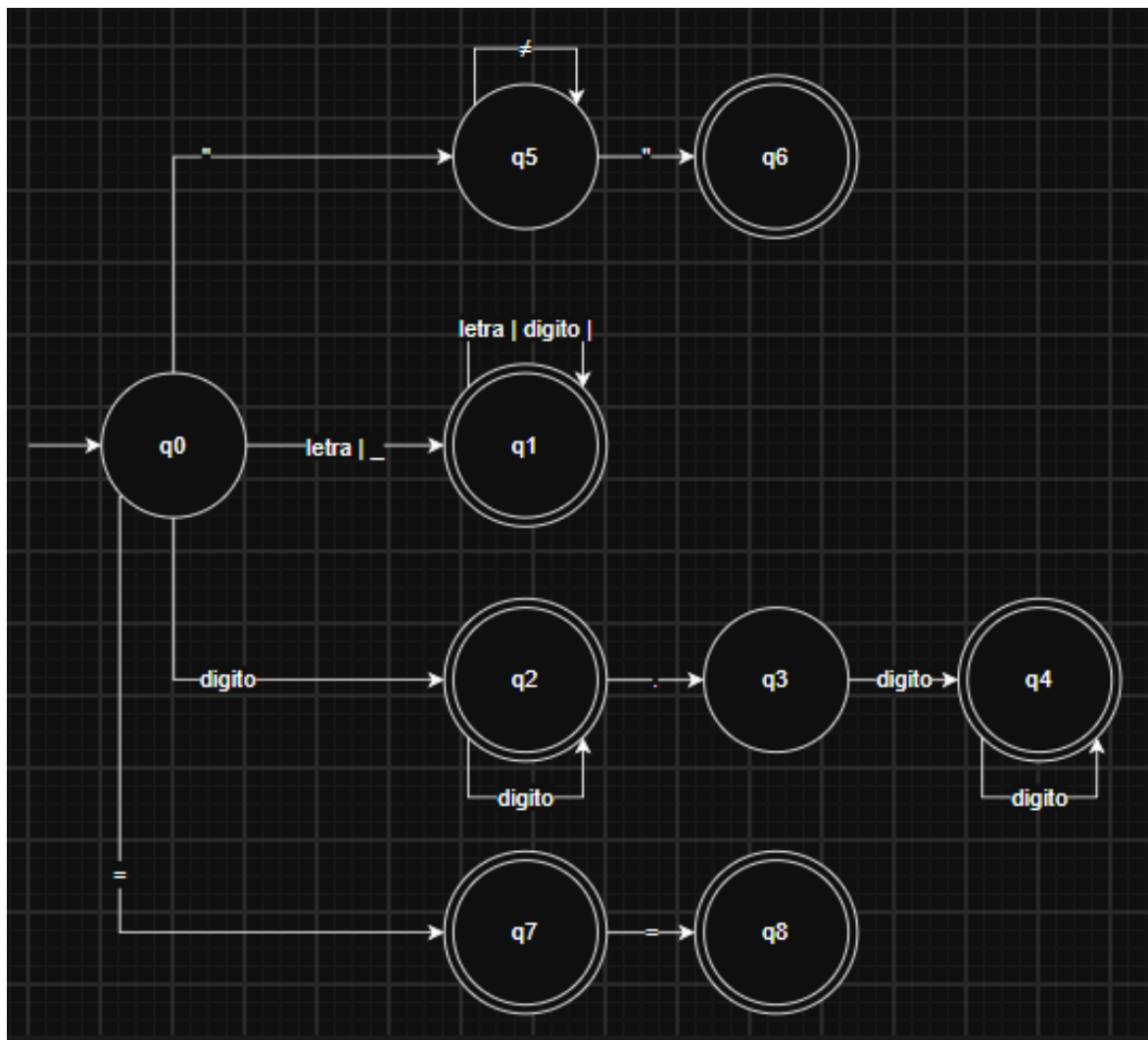
4. Cadenas de Texto Literales (CADENA)

Inicia con el símbolo literal de las comillas dobles, seguido de cero o más repeticiones de cualquier carácter del alfabeto aceptado que no sea una comilla doble (para evitar cierres prematuros), y culmina obligatoriamente con unas comillas dobles de cierre.

$$'"' \cdot \{ \text{Cualquier carácter excepto } "' \}^* \cdot "' = "' \{ '' \}^* '"$$

5. Diagrama del Autómata Finito

Figura 01. Diagrama del Autómata Finito creado por nosotros



Fuente. Elaboración propia diseñado en Draw.io

6. Analizador Léxico

El análisis léxico del compilador craft se ha implementado en C++ delegando la generación del autómata a la herramienta Flex (Fast Lexical Analyzer Generator). Flex toma como entrada el archivo de especificación `lexer.l`, compila las expresiones regulares internamente convirtiéndolas primero en un Autómata Finito No Determinista (NFA) mediante la construcción de Thompson, y luego lo transforma en un Autómata Finito Determinista (DFA) equivalente optimizado. Finalmente, este DFA se exporta como código fuente C++ (`lexer.cpp`) que contiene la función principal `yylex()`.

Para ilustrar el funcionamiento de esta fase, se presenta el siguiente flujo de transformación léxica:

Entrada (Código Fuente):

```
entero x = 10;
```

Salida (Flujo de Tokens Reconocidos):

```
TOKEN ENTERO    (Lexema: "entero")
TOKEN ID        (Lexema: "x")
TOKEN ASIGNAR   (Lexema: "=")
TOKEN NUM_ENTERO (Lexema: "10")
TOKEN PUNTOCOMA (Lexema: ";")
```

1. Configuración y Expresiones Regulares Base

El archivo `lexer.l` inicia con un bloque de código C++ delimitado por `%{` y `%}`. Este código es inyectado directamente en el analizador generado. Aquí se incluyen las cabeceras necesarias, el enlace con el analizador sintáctico (`parser.hpp`) y se definen las variables globales para el rastreo posicional preciso.

```
%{
#include "ast.h"
#include "parser.hpp"
#include <string>

// Contadores globales para reportar errores y construir el AST visualmente
int current_line = 1;
int current_col = 1;

// Acción ejecutada en CADA token reconocido: Guarda la metadata para los reportes
void add_tok(const std::string& name) {
    lexical_tokens.push_back({name, yytext, current_line, current_col});
    current_col += yyleng;
}
%}

%option noyywrap
```

Nota. La función personalizada `add_tok` es vital para la interfaz visual del proyecto. Antes de retornar cualquier token al Parser, esta función almacena el nombre del token, su lexema exacto (`yytext`), y su ubicación (`current_line`, `current_col`), actualizando dinámicamente la columna sumando `yyleng` (la longitud del lexema detectado).

2. Reglas de Producción Léxica

Las reglas léxicas definen el patrón a buscar y el código C++ a ejecutar. Flex utiliza el principio de Longitud Máxima (Maximal Munch): el escáner intentará leer tantos caracteres consecutivos como sea posible mientras la expresión regular siga coincidiendo. Si dos reglas coinciden con la misma longitud de cadena, se prefiere la regla que fue definida primero en el archivo.

```
%%
// 1. PALABRAS RESERVADAS (Deben ir primero por el principio de
// prioridad)
"entero"          { add_tok("ENTERO"); yylval.str = new std::string(yytext);
return ENTERO_T; }
"funcion_matematica" { add_tok("FUNCION_MAT"); yylval.str = new
std::string(yytext); return FUNCION_MAT_T; }

// 2. NÚMEROS Y CADENAS
[0-9]+           { add_tok("NUM_ENTERO"); yylval.str = new
std::string(yytext); return NUM_ENTERO; }
"\"[^\"]*"        { add_tok("LITERAL_CADENA");
std::string txt = yytext;
// Extraemos el texto ignorando las comillas literales
yylval.str = new std::string(txt.substr(1, txt.length()-2));
return CADENA; }

// 3. IDENTIFICADORES (Una letra o guion bajo seguido de 0 o mas
// caracteres validos)
[a-zA-Z_][a-zA-Z0-9_]* { add_tok("IDENTIFICADOR"); yylval.str = new
std::string(yytext); return ID; }
```

Nota. El atributo `yylval` actúa como el puente de datos entre el Lexer (Flex) y el Parser (Bison). Cuando reconocemos un identificador o número, creamos una copia de su contenido bruto almacenándolo dinámicamente en memoria (`new std::string(yytext)`) y lo asignamos a `yylval.str` antes de retornar el token.

3. Tabla Completa de Tokens

A continuación se presenta el mapeo completo de las expresiones regulares hacia sus respectivos tokens definidos en la gramática de craft:

Tabla 07. Tabla Completa de Tokens

Categoría	Patrón Regular	Acción (Token/Ignorar)
Palabras reservadas	"entero", "derivar", "leer"...	ENTERO_T, DERIVAR, LEER...
Literales Enteros	[0-9]+	NUM_ENTERO
Cadenas de texto	"\"[^\"]*"\"	CADENA
Identificadores	[a-zA-Z_][a-zA-Z0-9_]*	ID
Operadores aritm.	+, *, ^	'+', '*', '^'
Asignación	=	'='
Delimitadores	(,), :, ,	'(', ')', ':', ','
Retornos de carro	\r	Ignorado (skip)
Espacios y Tabs	[\t]+	Ignorado (solo suma yyleng a la columna)
Saltos de línea	\n	Ignorado (aumenta current_line y reinicia current_col)

Fuente. Elaboración propia

4. Detección y Manejo de Errores Léxicos

Una característica robusta del analizador léxico de craft es su capacidad de recuperarse ante caracteres no reconocidos. Para lograr esto, se utiliza la regla universal de un punto (.) al final del archivo lexer.l. Puesto que Flex da prioridad a las reglas definidas antes o a las de mayor longitud, esta regla de fallback (comodín) solo atrapa caracteres inválidos que no pertenecen al alfabeto del lenguaje (por ejemplo @, # o ~).

```
// ... reglas anteriores ...

\n      { current_line++; current_col = 1; }

// Regla general de manejo de errores (Cualquier caracter no capturado)
.      { add_tok("ERROR"); total_errors++; }

%%
```

Nota. Esta técnica permite al compilador advertir al programador registrando el error directamente en la tabla de métricas visuales (**total_errors++**) e insertando el token "ERROR" en el flujo sin colgarse ni abortar la ejecución

abruptamente. Esto asegura que el proceso continúe y permite al analizador reportar múltiples errores en una sola pasada de compilación.

7. Gramática del Lenguaje

Para la demostración pondremos veremos una versión inicial con ambigüedad.

Esta es la gramática base de craft. Aún contiene recursividad por la izquierda en las expresiones matemáticas y ambigüedad en las listas de sentencias y condicionales.

```

Programa -> LLAVEIZQ ListaSentencias LLAVEDER

ListaSentencias -> Sentencia ListaSentencias
                  | Sentencia

Sentencia -> Asignacion
            | Lectura
            | Escritura
            | Condicional
            | CicloMientras

Asignacion -> ID ASIGNAR Expresion PUNTOCOMA

Lectura -> leer PARENIZQ ID PARENDER PUNTOCOMA

Escritura -> imprimir PARENIZQ Expresion PARENDER PUNTOCOMA

Condicional -> si PARENIZQ Condicion PARENDER LLAVEIZQ ListaSentencias
LLAVEDER
              | si PARENIZQ Condicion PARENDER LLAVEIZQ ListaSentencias LLAVEDER
sino LLAVEIZQ ListaSentencias LLAVEDER

CicloMientras -> mientras PARENIZQ Condicion PARENDER LLAVEIZQ ListaSentencias
LLAVEDER

Condicion -> Expresion OperadorRelacional Expresion

OperadorRelacional -> EQUALS
                    | NEQUALS
                    | MAYOR
                    | MENOR

Expresion -> Expresion MAS Termino
            | Expresion MENOS Termino
            | Termino

Termino -> Termino POR Factor
          | Termino DIV Factor
          | Factor

```



```
Factor -> Factor POTENCIA Base
      | Base
```

```
Base -> ID
      | NUM_ENTERO
      | NUM_DECIMAL
      | CADENA
      | PARENIZQ Expresion PARENDER
```

Nota. No Terminales: Programa, ListaSentencias, Sentencia, Asignacion, Lectura, Escritura, Condicional, CicloMientras, Condicion, OperadorRelacional, Expresion, Termino, Factor, Base.

Terminales: (Tus tokens en mayúsculas y palabras reservadas en minúsculas).

8. Eliminación de Ambigüedad

Aplicamos las reglas para transformar nuestra gramática inicial en una gramática LL(1) apta para un analizador sintáctico descendente. Utilizaremos el símbolo ϵ (épsilon) para representar la producción vacía.

A. Factorización por la Izquierda

Buscamos producciones que comiencen con los mismos símbolos y bifurcamos el camino usando un "Prima" (').

1. Factorización de ListaSentencias:

Ambas opciones inician con Sentencia.

```
ListaSentencias -> Sentencia ListaSentenciasPrima
ListaSentenciasPrima -> ListaSentencias |  $\epsilon$ 
```

2. Factorización de Condicional (Dangling Else):

Ambas producciones comparten si (Condicion) { ListaSentencias }.

```
Condicional -> si PARENIZQ Condicion PARENDER LLAVEIZQ ListaSentencias
LLAVEDER CondicionalPrima
CondicionalPrima -> sino LLAVEIZQ ListaSentencias LLAVEDER
                  |  $\epsilon$ 
```

B. Eliminación de Recursividad por la Izquierda

Un analizador LL(1) no soporta reglas donde un No Terminal se llama a sí mismo inmediatamente a la izquierda (ej. A(alpha)). Lo pasamos a recursividad por la derecha.

1. Eliminación en Expresión (Suma y Resta):

```
Expresion -> Termino ExpresionPrima
ExpresionPrima -> MAS Termino ExpresionPrima
                | MENOS Termino ExpresionPrima
                | ε
```

2. Eliminación en Término (Multiplicación y División):

```
Termino -> Factor TerminoPrima
TerminoPrima -> POR Factor TerminoPrima
              | DIV Factor TerminoPrima
              | ε
```

3. Eliminación en Factor (Potencia):

```
Factor -> Base FactorPrima
FactorPrima -> POTENCIA Base FactorPrima
            | ε
```

C. Conversión LL(1) (Gramática Final Optimizada)

Al unir todos los pasos, esta es la gramática de craft lista para programar tu Analizador Sintáctico. No tiene ambigüedad ni recursividad izquierda.

```
Programa -> LLAVEIZQ ListaSentencias LLAVEDER

ListaSentencias -> Sentencia ListaSentenciasPrima
ListaSentenciasPrima -> ListaSentencias
                    | ε

Sentencia -> Asignacion
          | Lectura
          | Escritura
          | Condicional
          | CicloMientras

Asignacion -> ID ASIGNAR Expresion PUNTOCOMA

Lectura -> leer PARENIZQ ID PARENDER PUNTOCOMA
```

```

Escritura -> imprimir PARENIZQ Expresion PARENDER PUNTOCOMA

Condicional -> si PARENIZQ Condicion PARENDER LLAVEIZQ ListaSentencias
LLAVEDER CondicionalPrima
CondicionalPrima -> sino LLAVEIZQ ListaSentencias LLAVEDER
| ε

CicloMientras -> mientras PARENIZQ Condicion PARENDER LLAVEIZQ ListaSentencias
LLAVEDER

Condicion -> Expresion OperadorRelacional Expresion

OperadorRelacional -> EQUALS | NEQUALS | MAYOR | MENOR

Expresion -> Termino ExpresionPrima
ExpresionPrima -> MAS Termino ExpresionPrima
| MENOS Termino ExpresionPrima
| ε

Termino -> Factor TerminoPrima
TerminoPrima -> POR Factor TerminoPrima
| DIV Factor TerminoPrima
| ε

Factor -> Base FactorPrima
FactorPrima -> POTENCIA Base FactorPrima
| ε

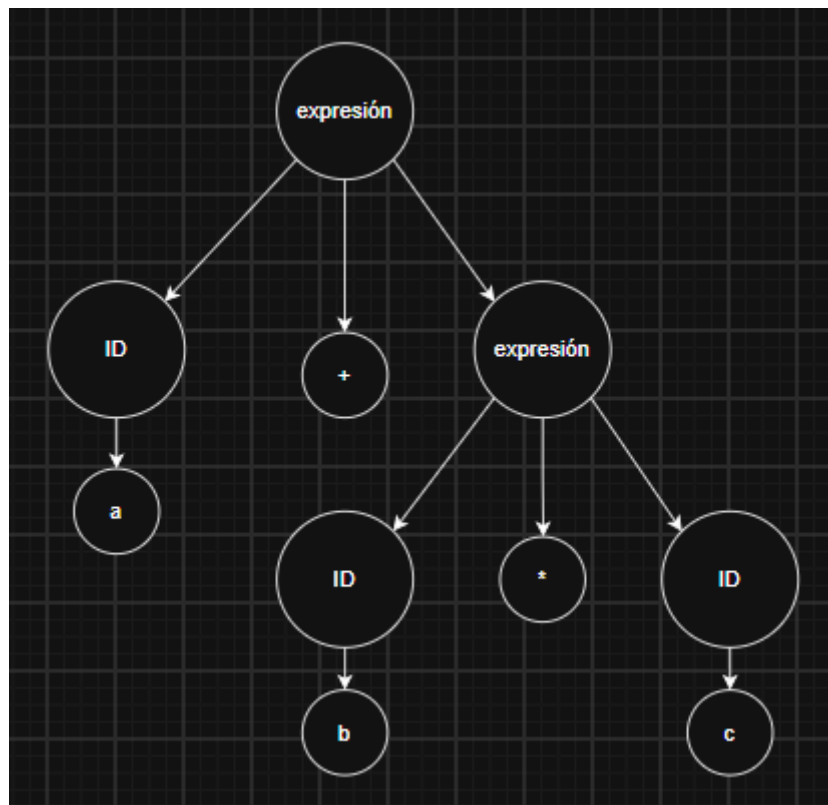
Base -> ID
| NUM_ENTERO
| NUM_DECIMAL
| CADENA
| PARENIZQ Expresion PARENDER

```

Nota. Esto representa la gramática formal definitiva del lenguaje craft, estructurada específicamente como una Gramática LL(1) (lectura de izquierda a derecha, derivación por la izquierda, con un token de anticipación), y ahora es una gramática determinista pura y sin ambigüedades. Esto significa que el compilador ya no tiene que "adivinar" ni retroceder (backtracking) para entender el código fuente. Al leer un solo token, sabe exactamente qué regla aplicar, garantizando que la construcción del Árbol de Sintaxis Abstracta (AST) sea 100% predecible, rápida y estructuralmente correcta en la fase de Análisis Sintáctico.

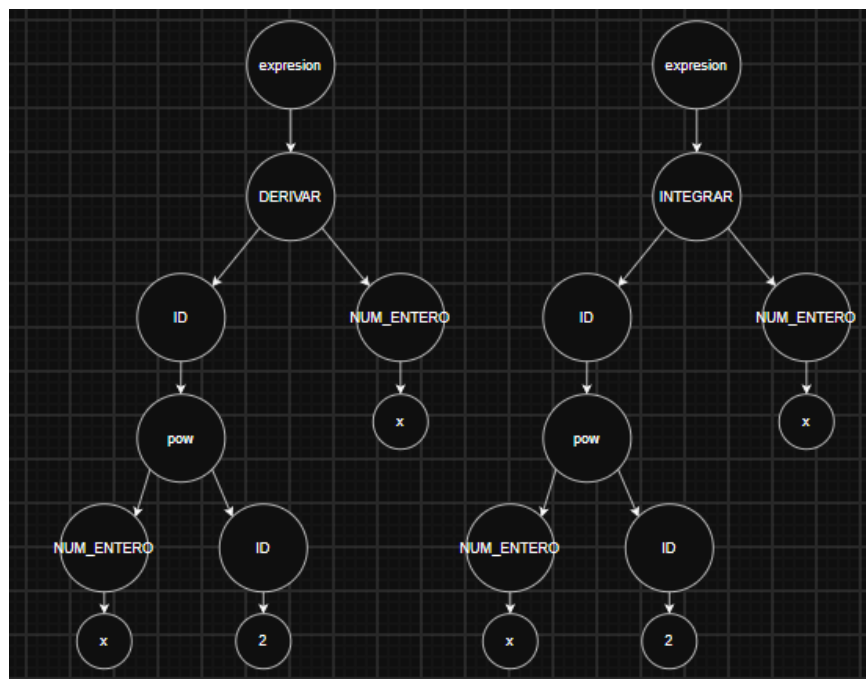
9. Árbol Sintáctico

Figura 02. Árbol Sintáctico del ejercicio prueba1



Fuente. Elaboración propia

Figura 03. Árbol Sintáctico del ejercicio prueba2



Fuente. Elaboración propia

10. Analizador Sintáctico Implementado

Una gramática regular (como la usada en el lexer) no puede contar ni anidar estructuras, como verificar que los paréntesis o las llaves estén correctamente balanceados. El análisis sintáctico (o parser) sí puede hacerlo mediante una Gramática Libre de Contexto. En nuestro lenguaje craft, el análisis sintáctico permite:

- Validar que la sintaxis general es correcta (por ejemplo, que una declaración entero x termine correctamente en un punto y coma ;).
- Capturar la precedencia y asociatividad de los operadores matemáticos (asegurando que * y / se evalúen antes que + y -).
- Construir una representación en memoria abstracta (AST) para manipular el programa más fácilmente en las siguientes fases del compilador.

1. Implementación con Bison: parser.y

Para construir el analizador sintáctico de craft, empleamos Bison, que es un generador de parsers de tipo LALR(1) (Look-Ahead Left-to-Right, Rightmost derivation with 1 token lookahead). Bison toma nuestra CFG escrita en el archivo parser.y y genera automáticamente una máquina de estados (Pushdown Automaton) en lenguaje C/C++.

2. Funcionamiento Interno (Shift/Reduce)

El parser generado por Bison funciona evaluando la entrada mediante dos operaciones principales sobre una pila sintáctica:

- **Shift (Desplazamiento):** Lee un token de la entrada y lo empuja a la pila sintáctica, pasando a un nuevo estado.
- **Reduce (Reducción):** Cuando la cima de la pila coincide con el lado derecho de una producción (por ejemplo, ID '=' expresion), saca esos elementos de la pila y empuja el lado izquierdo (el No Terminal, como asignacion). En este preciso momento se ejecuta el código C++ asociado (la acción semántica), que en nuestro caso crea un nodo del AST.

3. Precedencia y Tipos

```
%union {
    std::string* str;
    ASTNode* node;
    BlockNode* block;
    DataType type;
}
%token <str> ID NUM_ENTERO CADENA ENTERO_T FUNCION_MAT_T
%token LEER IMPRIMIR DERIVAR INTEGRAR_INDEFINIDA

%left '+' '-'
%left '*' '/'
%right '^'
```

Nota. En esta sección de declaraciones de Bison, se define un %union que permite a los tokens y no terminales transportar distintos tipos de datos (como strings, tipos de datos y punteros a nodos del AST). Además, se establecen las reglas de precedencia y asociatividad (%left y %right) para los operadores aritméticos, lo que elimina la ambigüedad en expresiones matemáticas complejas propias de las funciones del lenguaje (como la integración y derivación).

4. Gramática y Construcción del AST

```
expresion
: expresion '+' expresion { $$ = new BinaryOpNode("+", $1, $3); }
| expresion '*' expresion { $$ = new BinaryOpNode("*", $1, $3); }
| expresion '^' expresion { $$ = new BinaryOpNode("^", $1, $3); }
| DERIVAR '(' ID ',' ID ')' { $$ = new MathNativeNode("derivar", *$3, *$5); }
| INTEGRAR_INDEFINIDA '(' ID ',' ID ')' { $$ = new
MathNativeNode("integrar_indefinida", *$3, *$5); }
| NUM_ENTERO { $$ = new LiteralNode(*$1, DataType::ENTERO); }
| ID { $$ = new VariableNode(*$1); }
;
```

Nota. Este fragmento muestra la gramática libre de contexto aplicada a las expresiones. Las llaves { } a la derecha de cada producción contienen las acciones semánticas ejecutadas durante una operación Reduce. Como se observa, cada regla instancia de manera dinámica una clase específica (BinaryOpNode, MathNativeNode, etc.), enlazando los sub-nodos (\$1, \$3) para ir construyendo de abajo hacia arriba (bottom-up) el Árbol de Sintaxis Abstracta (AST) en memoria.

5. Ejercicios implementados obteniendo el AST en el programa

Para la obtención del AST en nuestro programa se probaron los siguientes ejercicios

Ejercicio prueba1:

```
entero a;
entero b;
entero c;
entero respuesta;
leer(a);
leer(b);
leer(c);
respuesta = a + b * c;
imprimir("El resultado es: ", respuesta);
```

Figura 04. AST obtenido del ejercicio1

```
=== AST (forma jerarquica) ===
PROGRAMA (programa) [linea 1, col 1]
|-- DECLARACION (a) [linea 1]
|   |-- TIPO (ENTERO)
|-- DECLARACION (b) [linea 2]
|   |-- TIPO (ENTERO)
|-- DECLARACION (c) [linea 3]
|   |-- TIPO (ENTERO)
|-- DECLARACION (respuesta) [linea 4]
|   |-- TIPO (ENTERO)
|-- LEER [linea 5]
|   |-- ID (a)
|-- LEER [linea 6]
|   |-- ID (b)
|-- LEER [linea 7]
|   |-- ID (c)
|-- ASIGNACION (respuesta) [linea 8]
|   |-- ID (respuesta)
|   |-- OP_BINARIA (+) [linea 8]
|       |-- IDENTIFICADOR (a) [linea 8]
|       |-- OP_BINARIA (*) [linea 8]
|           |-- IDENTIFICADOR (b) [linea 8]
|           |-- IDENTIFICADOR (c) [linea 8]
|-- ESCRIBIR [linea 9]
|   |-- CADENA (El resultado es: )
|   |-- ID (respuesta)

Presiona Enter para continuar...[]
```

Fuente. Elaboración del Ast propia obtenida de nuestro programa

Ejercicio prueba2:

```

entero x;
funcion_matematica f;
funcion_matematica d;
funcion_matematica i;
x = 5;
f = x ^ 2;
d = derivar(f, x);
i = integrar_indefinida(f, x);
imprimir("La derivada es: ", d);
imprimir("La integral es: ", i);

```

Figura 05. AST obtenido del ejercicio2

```

=====
Opcion: 4

=== AST (forma jerarquica) ===
PROGRAMA (programa) [linea 1, col 1]
|-- DECLARACION (x) [linea 1]
|   |-- TIPO (ENTERO)
|-- DECLARACION (f) [linea 2]
|   |-- TIPO (FUNCION_MAT)
|-- DECLARACION (d) [linea 3]
|   |-- TIPO (FUNCION_MAT)
|-- DECLARACION (i) [linea 4]
|   |-- TIPO (FUNCION_MAT)
|-- ASIGNACION (x) [linea 5]
|   |-- ID (x)
|   |-- LITERAL (5) [linea 5]
|-- ASIGNACION (f) [linea 6]
|   |-- ID (f)
|   |-- OP_BINARIA (^) [linea 6]
|       |-- IDENTIFICADOR (x) [linea 6]
|       |-- LITERAL (2) [linea 6]
|-- ASIGNACION (d) [linea 7]
|   |-- ID (d)
|   |-- NATIVA (derivar) [linea 7]
|       |-- ID_FUNCION (f)
|       |-- ID_VARIABLE (x)
|-- ASIGNACION (i) [linea 8]
|   |-- ID (i)
|   |-- NATIVA (integrar_indefinida) [linea 8]
|       |-- ID_FUNCION (f)
|       |-- ID_VARIABLE (x)
|-- ESCRIBIR [linea 9]
|   |-- CADENA (La derivada es: )
|   |-- ID (d)
|-- ESCRIBIR [linea 10]
|   |-- CADENA (La integral es: )
|   |-- ID (i)

```

Fuente. Elaboración del Ast propia obtenida de nuestro programa

11. Tabla de Símbolos

Para la obtención de la tabla de símbolos usaremos los ejercicios que probamos antes.

Figura 06. Tabla de símbolos del ejercicio prueba1

```

=== TABLA DE SIMBOLOS ===

```

NOMBRE	TIPO	VALOR	DIRECCION	AMBITO	LINEA
respuesta	ENTERO	<entrada>	7	0	4
c	ENTERO	<entrada>	6	0	3
b	ENTERO	<entrada>	5	0	2
a	ENTERO	<entrada>	4	0	1

Fuente. Tabla obtenida de nuestro programa

Figura 07. Tabla de símbolos del ejercicio prueba2:

```

=== TABLA DE SIMBOLOS ===

```

NOMBRE	TIPO	VALOR	DIRECCION	AMBITO	LINEA
i	FUNCION_MAT	<entrada>	3	0	4
d	FUNCION_MAT	<entrada>	2	0	3
f	FUNCION_MAT	<entrada>	1	0	2
x	ENTERO	<entrada>	0	0	1

Presiona Enter para continuar...

Fuente. Tabla obtenida de nuestro programa

12. Analizador Semántico

El análisis semántico es la fase del compilador encargada de verificar que el código fuente, además de estar bien escrito (sintaxis), tenga sentido y cumpla con las reglas del lenguaje craft. Para esto, el analizador recorre el Árbol de Sintaxis Abstracta (AST) generado en la fase anterior y se apoya en una Tabla de Símbolos para llevar el registro de los identificadores y sus propiedades.

1. Validaciones Implementadas

Variables declaradas: Se verifica que todo identificador (ID) haya sido declarado previamente antes de ser utilizado en una expresión o asignación. Si se intenta leer, imprimir o asignar un valor a una variable que no existe en la Tabla de Símbolos, se detiene la ejecución.

Variables inicializadas: Garantiza que una variable contenga un valor válido antes de participar en una operación matemática o ser enviada a funciones nativas como derivar o integrar_indefinida.

Compatibilidad de tipos: Asegura que las operaciones y asignaciones se realicen entre tipos de datos coherentes. En craft, un tipo entero no puede recibir un valor de tipo texto (cadena), y los operadores aritméticos (+, -, *, /, ^) exigen operandos numéricos o funciones matemáticas compatibles.

Conversión de tipos (Casting implícito/explicito): Maneja la promoción de tipos seguros de forma automática. Por ejemplo, si se suma un entero con un decimal, el analizador semántico permite la operación promoviendo el resultado a decimal para no perder precisión.

Número de parámetros: Valida que las funciones nativas o rutinas reciban exactamente la cantidad y el tipo de argumentos que esperan. Por ejemplo, la función derivar(ID, ID) exige estrictamente dos parámetros; pasar un número distinto de argumentos lanzará una alerta.

2. Ejemplo de Validación Semántica

El siguiente fragmento muestra cómo el compilador de craft atrapa un error en la compatibilidad de tipos durante la asignación.

Código de entrada en craft:

```
entero x;  
x = "hola";
```

Salida del Compilador (Consola):

```
Error semántico: Tipos incompatibles. No se puede asignar un valor de tipo 'texto'  
a la variable 'x' de tipo 'entero'.
```

En nuestra arquitectura de clases en C++, esta validación se ejecuta recorriendo los nodos del AST (como AssignmentNode o BinaryOpNode). Cada nodo tiene un método que verifica sus tipos internos. Por ejemplo, al evaluar un AssignmentNode, el compilador consulta la Tabla de Símbolos para obtener el DataType del ID izquierdo y lo compara con el tipo de retorno evaluado de la expresión derecha. Si los tipos difieren y no existe una regla de conversión válida, se incrementa el contador de errores y se reporta el fallo.

13. Código Intermedio

Para la obtención de Código Intermedio usaremos los ejercicios que probamos antes

Figura 08. Código Intermedio del ejercicio prueba1

```

=== CODIGO INTERMEDIO (cuadрупlos) ===
#  OPERADOR      ARG1      ARG2      RESULTADO
0  PROGRAMA_INICIO
1  DECLARE      ENTERO      a
2  DECLARE      ENTERO      b
3  DECLARE      ENTERO      c
4  DECLARE      ENTERO      respuesta
5  READ
6  READ
7  READ
8  *            b          c          t1
9  +            a          t1          t0
10 =            t0          respuesta
11 WRITE      "El resultado es: "
12 WRITE      respuesta
13 PROGRAMA_FIN

Presiona Enter para continuar...

```

Fuente. Código Intermedio obtenido de nuestro programa

Figura 09. Código Intermedio del ejercicio prueba2

```

=== CODIGO INTERMEDIO (cuadрупlos) ===
#  OPERADOR      ARG1      ARG2      RESULTADO
0  PROGRAMA_INICIO
1  DECLARE      ENTERO      x
2  DECLARE      FUNCION_MAT  f
3  DECLARE      FUNCION_MAT  d
4  DECLARE      FUNCION_MAT  i
5  =            5            x
6  ^            x            2          t2
7  =            t2            f
8  derivar      f            x          t3
9  =            t3            d
10 integrar_indefinida f      x          t4
11 =            t4            i
12 WRITE      "La derivada es: "
13 WRITE      d
14 WRITE      "La integral es: "
15 WRITE      i
16 PROGRAMA_FIN

```

Fuente. Código Intermedio obtenido de nuestro programa

14. Optimización de Código

Para la obtención de la Optimización de Código usaremos los ejercicios que probamos antes.

Figura 10. Optimización de Código del ejercicio prueba1

```
=== CODIGO OPTIMIZADO ===
```

Nota. Para este código al ser básico no hubo optimización

Figura 11. Optimización de Código del ejercicio prueba2

```
=== CODIGO OPTIMIZADO ===
```

Nota. Para este código al tener datos estáticos no hubo optimización

15. Generación de Código Final

Para la obtención de Código Final usaremos los ejercicios que probamos antes, cabe recalcar que se generó en Código C++.

Figura 12. Código Final en c++ del ejercicio prueba1

```
int main() {
    int a;
    int b;
    int c;
    int respuesta;
    std::cin >> a;
    std::cin >> b;
    std::cin >> c;
    respuesta = (a + (b * c));
    std::cout << "El resultado es: " << respuesta << std::endl;

    return 0;
}

Presiona Enter para continuar...
```

Fuente. Código Final obtenido de nuestro programa

Figura 13. Código Final en c++ del ejercicio prueba2

```
int main() {
    int x;
    CRAFTFunction f;
    CRAFTFunction d;
    CRAFTFunction i;
    x = 5;
    f = pow(x, 2);
    d = CRAFTMathCore::derivar(f, "x");
    i = CRAFTMathCore::integrar_indefinida(f, "x");
    std::cout << "La derivada es: " << d << std::endl;
    std::cout << "La integral es: " << i << std::endl;

    return 0;
}

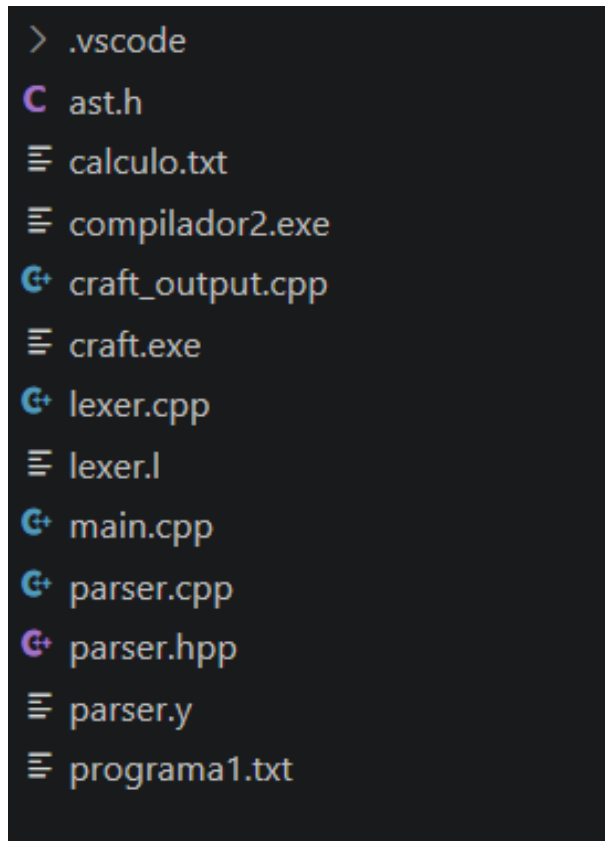
Presiona Enter para continuar...
```

Fuente. Código Final obtenido de nuestro programa

16. Implementación

La fase de implementación del compilador craft consiste en la traducción de los modelos teóricos (autómatas, gramáticas libres de contexto y reglas semánticas) a código ejecutable. Como se evidencia en la estructura de archivos del proyecto craft, se ha optado por un ecosistema de herramientas estándar en el desarrollo de compiladores, garantizando modularidad y eficiencia.

Figura 14. Archivos usado en este proyecto



Fuente. Archivos obtenidos de nuestro programa

Lenguaje Utilizado

C++:

Es el único lenguaje de programación empleado para el núcleo del compilador (evidenciado por los archivos .cpp, .h y .hpp). Se eligió C++ por su alto rendimiento, el control directo sobre la memoria y, fundamentalmente, por su soporte para el paradigma de Programación Orientada a Objetos (POO). Esta característica es vital para la construcción del Árbol de Sintaxis Abstracta (AST), permitiendo modelar cada estructura del lenguaje (asignaciones, expresiones, bucles) como nodos de clases derivadas (definidas en ast.h), facilitando así el recorrido y la evaluación semántica.

Herramientas Utilizadas

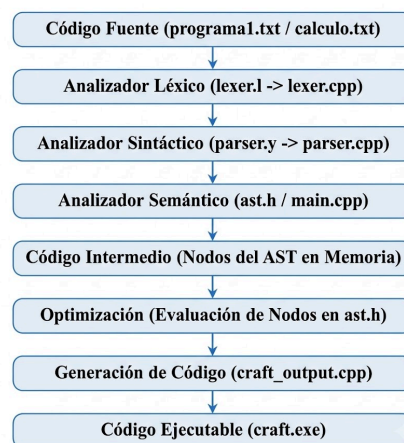
Para la automatización y gestión del código, el proyecto se apoya en el siguiente conjunto de herramientas especializadas:

- **Flex (Generador de Analizador Léxico):** Utilizado a través del archivo de definición `lexer.l`. Flex toma nuestras expresiones regulares y tokens, y genera automáticamente el código en C++ (`lexer.cpp`) correspondiente al autómata finito determinista. Este módulo se encarga de leer el código fuente (por ejemplo, `programa1.txt`) y convertirlo en un flujo de tokens.
- **Bison (Generador de Analizador Sintáctico):** Implementado a través del archivo `parser.y`. Bison procesa nuestra gramática libre de contexto LALR(1) y genera el autómata de pila en C++ (`parser.cpp` y `parser.hpp`). Trabaja en conjunto con la salida de Flex para validar la estructura del programa y construir dinámicamente el AST en memoria.
- **Visual Studio Code (VS Code):** El Entorno de Desarrollo Integrado (IDE) elegido para gestionar el proyecto, como lo indica la presencia del directorio de configuración `.vscode`. Se seleccionó por su ligereza, su amplia compatibilidad con extensiones para C/C++ y su terminal integrada, la cual agiliza el ciclo de compilación de los archivos fuente hacia el ejecutable final

17.Arquitectura del Sistema

La arquitectura del compilador craft sigue un modelo secuencial en fases, donde la salida de cada etapa sirve como entrada para la siguiente, nuestro proyecto adopta un enfoque práctico de traducción (transpilación) utilizando C++ como lenguaje puente para la generación del ejecutable final.

Figura 15. Arquitectura de nuestro sistema del proyecto



Fuente. Elaboración propia

Descripción de las Fases

Código Fuente (.txt): Es el archivo de entrada escrito por el usuario utilizando la sintaxis de craft (por ejemplo, programa1.txt), el cual contiene las palabras reservadas, declaraciones y funciones matemáticas.

Analizador Léxico (lexer.l): Flex lee el código fuente y lo divide en componentes léxicos (Tokens) definidos mediante expresiones regulares, descartando espacios y comentarios.

Analizador Sintáctico (parser.y): Bison recibe los tokens y valida que cumplan con la gramática libre de contexto LALR(1). Durante las operaciones de reducción, ejecuta acciones semánticas que instancian las clases definidas en nuestro código.

Analizador Semántico (ast.h / main.cpp): Se encarga de verificar el significado del código. Valida la declaración de variables en la Tabla de Símbolos, verifica la compatibilidad de tipos (ej. no sumar un texto con un entero) y confirma la correcta cantidad de parámetros en funciones como derivar().

Código Intermedio (AST en Memoria): En lugar de generar un archivo de texto intermedio (.ir), craft construye un Árbol de Sintaxis Abstracta (AST) en la memoria dinámica. Las clases base y derivadas de ast.h representan la estructura jerárquica y lógica del programa.

Optimización: Fase donde se puede simplificar el árbol. En nuestro proyecto, esto ocurre al evaluar expresiones constantes directamente en los nodos del AST (por ejemplo, resolver una suma estática de $2 + 3$ antes de generar el código final) para hacer la ejecución más eficiente.

Generación de Código (craft_output.cpp): El compilador recorre el AST ya validado y lo traduce a código fuente en C++. Esta estrategia aprovecha la potencia de C++ para manejar el tipado estricto y las funciones matemáticas requeridas por el usuario.

Código Ejecutable (craft.exe): El archivo final resultante. El código C++ generado (craft_output.cpp) es compilado (usualmente de forma automática por el sistema a través del compilador subyacente del IDE o MinGW) para producir el binario .exe que el usuario final puede ejecutar nativamente en su máquina.

18. Interfaz del Programa

Figura 16. Primera fase de programa

```

===== FASE 1: ENTRADA DE CODIGO =====
Archivo actual: (ninguno)
-----
1. Abrir archivo
2. Escribir codigo aqui mismo
3. Ingresar (Continuar a Fase 2)
0. Salir
=====
Opcion: █

```

Fuente. Imagen obtenida de nuestro programa

Figura 17. Ingresamos un archivo con nuestro lenguaje

```

===== FASE 1: ENTRADA DE CODIGO =====
Archivo actual: calculo.txt
-----
1. Abrir archivo
2. Escribir codigo aqui mismo
3. Ingresar (Continuar a Fase 2)
0. Salir
=====
Opcion: █

```

Fuente. Imagen obtenida de nuestro programa

Figura 18. Demás características una vez reconocido el archivo

```

===== FASE 2: COMPILADOR =====
Archivo procesando: calculo.txt
-----
1. Analisis lexico
2. Analisis sintactico
3. Analisis semantico
4. Mostrar AST
5. Mostrar Tabla de Simbolos
6. MostrarCodigo Intermedio
7. MostrarCodigo Optimizado
8. GenerarCodigo C++
9. Compilar automaticamente el C++ (g++)
10. Volver a ingresar codigo (Regresar a Fase 1)
0. Salir
=====
Opcion: █

```

Fuente. Imagen obtenida de nuestro programa

19. Resultados

Tras la integración de las herramientas y la codificación de la arquitectura en C++, se obtuvieron los siguientes resultados prácticos:

Se logró implementar un compilador funcional para el lenguaje craft que cubre de manera exitosa todas las fases: desde el análisis léxico hasta la generación de código destino ejecutable mediante g++.

El analizador sintáctico y el Árbol de Sintaxis Abstracta (AST) demostraron procesar correctamente la jerarquía matemática y la precedencia de operadores. Esto se comprobó al evaluar programa1.txt, donde expresiones mixtas como $a + b * c$ fueron estructuradas y generadas sin ambigüedades.

Las pruebas con calculo.txt validaron la capacidad del compilador para manejar tipos de datos personalizados (funcion_matematica) y enrutar operaciones simbólicas (derivar, integrar_indefinida) hacia una clase núcleo generada en C++ (CRAFTMathCore).

El desarrollo del menú interactivo por consola permitió visualizar en tiempo real los artefactos internos del compilador (Tabla de Símbolos, Nodos del AST y Cuádruplos de Código Intermedio), demostrando que el estado de la memoria se gestiona de manera consistente durante el análisis.

Se comprobó la funcionalidad del método de optimización por "Plegado de Constantes" (Constant Folding) en la clase BinaryOpNode, logrando simplificar operaciones entre literales enteros antes de la generación del código final.

20. Conclusiones

Se logró desarrollar exitosamente el compilador craft, dotándolo de un conjunto de palabras reservadas netamente en español (como entero, leer, imprimir, derivar). Esto cumple el propósito fundamental de reducir la barrera idiomática, permitiendo a los estudiantes interiorizar la complejidad de la teoría de lenguajes y autómatas de forma práctica, directa y familiar.

La integración estructurada de las herramientas Flex y Bison demostró ser altamente eficiente. Se consolidó un frontend capaz de tokenizar correctamente el código fuente y validar su gramática, culminando con éxito en la construcción automatizada del Árbol de Sintaxis Abstracta (AST) en memoria.

Se construyó un validador semántico robusto gestionado a través de una Tabla de Símbolos dinámica en C++. Esta fase cumple a cabalidad con la verificación de reglas del lenguaje, asegurando que los identificadores existan previamente, validando la estricta compatibilidad de los tipos de datos y controlando la coherencia en el uso de las funciones matemáticas.

Se logró transformar exitosamente las instrucciones jerárquicas del programa en una representación intermedia estructurada (mediante el uso de cuádruplos y el AST). Esta abstracción cumplió su función como puente lógico indispensable para independizar el análisis del código de su posterior traducción hacia el código destino.

La arquitectura basada en nodos permitió aplicar rutinas de optimización directamente sobre el árbol. Se implementó con éxito la simplificación algebraica y la reducción de sentencias mediante el "Plegado de Constantes" (resolviendo operaciones estáticas en tiempo de compilación), sentando las bases estructurales para futuras implementaciones de eliminación de código muerto.

El ciclo de vida del compilador fue validado empíricamente a través de la ejecución de diferentes ejercicios de prueba (como programa1.txt y calculo.txt). Estos programas demostraron el correcto engranaje de todas las fases implementadas, logrando leer variables, procesar precedencia matemática y generar un archivo ejecutable final funcional.

21. Recomendaciones

Para superar la limitación de depender de un compilador secundario subyacente (GNU GCC / g++), se recomienda desarrollar a futuro un generador de código que emita directamente lenguaje ensamblador (por ejemplo, x86 o RISC-V) o integrar el proyecto con un framework como LLVM. Esto consolidaría al proyecto estructuralmente como un compilador puro y no como un transpilador hacia C++.

Con el fin de aumentar la capacidad de abstracción del lenguaje craft, se sugiere extender la gramática en Bison y la estructura jerárquica del AST para soportar la Programación Orientada a Objetos (POO). Implementar el manejo de clases, herencia y objetos permitiría a los estudiantes desarrollar software modular y más complejo.

Dado que el diseño actual opera bajo un flujo de memoria estática, se recomienda introducir el soporte para punteros y referencias. Al actualizar la Tabla de Símbolos y las reglas de validación semántica para permitir la asignación dinámica de memoria, los usuarios podrán construir estructuras de datos avanzadas como listas enlazadas o árboles.

Siendo el cálculo matemático la principal innovación del proyecto, el siguiente paso natural es potenciar las funciones de derivación e integración. Se recomienda expandir el algoritmo actual —centrado en polinomios simples— para que el analizador sea capaz de procesar algebraicamente expresiones complejas, funciones trigonométricas, logarítmicas y derivadas de orden superior.

Se sugiere seguir aprovechando el modelo de nodos del AST para implementar algoritmos agresivos de optimización, tales como la Eliminación de Código Muerto (Dead Code Elimination) y la Propagación de Copias. Esto garantizará que el código intermedio (o C++) generado sea lo más limpio y eficiente posible antes de ser entregado al compilador secundario para la generación del .exe.

22. Bibliografía

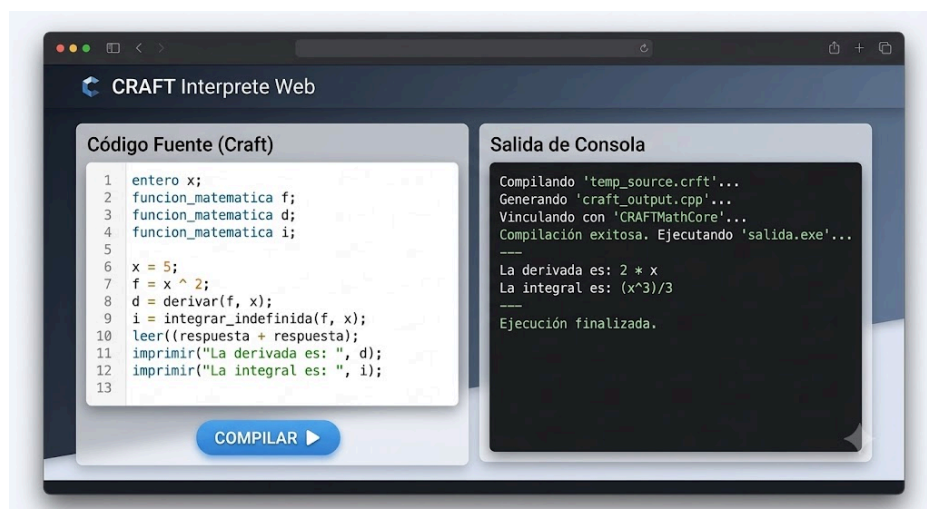
Aho, A., Lam, M., Sethi, R., and Ullman, J. D (2008). Compiladores. Principios, técnicas y herramientas (2da. Edición).

Louden, Kenneth (2004). Construcción de compiladores. Cengage Learning Latin America.

Nystrom, Robert (2021). Crafting interpreters. Genever Benning.

23. Anexo

Anexo 01. Imagen del boceto de una página web para probar el lenguaje craft



Fuente. Imagen generada con una IA para ver el boceto de la página web